

BuzzCalculus 的引擎

沒有後端，怎麼判答案、驗證明、決定你今天練什麼

一個純靜態的微積分訓練站：沒有伺服器、沒有帳號，所有運算在瀏覽器裡完成。本報告整理它的三個純函式引擎——答案判分、白話證明檢查、能力模型與抽題——以及把每一條承諾綁進持續整合的驗證文化。全部數字取自 2026 年 9 月 16 日的程式碼與驗證器輸出。

專案	<code>tudohuang / BuzzCalculus</code>
線上	<code>https://tudohuang.github.io/BuzzCalculus/</code>
原始碼	<code>https://github.com/tudohuang/BuzzCalculus</code>
規模	1,994 題·kernel 14,489 行·app.js 15,577 行·41 支驗證器·9 支 E2E

摘要

BuzzCalculus 把「練微積分」這件事拆成三個可以被驗證的問題：使用者打進來的答案對不對、使用者寫的證明有沒有跳步、今天應該練什麼。三個問題各有一個引擎，全部是純函式：輸入紀錄、題庫與時間，輸出資料，不碰 DOM、不碰儲存、不呼叫系統時鐘。這讓同一份程式碼能在瀏覽器裡跑，也能在 Node 的驗證器裡被逐題釘住。

判分引擎不做符號化簡，而是在刻意挑過的取樣點上比較兩個式子的值；不定積分比較的是相對基準點的增量，因此對積分常數不敏感。標準答案本身另有一支獨立驗算器把關：它從題幹推出一條與解題無關的數值路徑再和答案比，目前覆蓋 1,753 題 (88%)，不符數在 CI 裡必須為零。

白話證明引擎把中文證明逐行分成 13 種句型，對代數鏈在當前假設下隨機取點驗證，對定理引用比對規則字典；驗不了的三分之一(存在句、抽象函數)標黃而不假裝。每一題的參考證明都經過三種變異測試：刪一行、改一個符號、換方向，變異版必須紅在正確的那一行。

能力模型從作答紀錄回放 80 個技巧的熟練度，每個常數都能解釋：30 天的權重時間常數、相當於 6 次不確定作答的先驗、加權量 12 才顯示分數。規劃器輸出的不是題目而是「配方 + 為什麼」，抽題器抽不滿一定降級補滿並記錄降級路徑。

報告最後一部分是驗證文化：41 支驗證器、9 支以原生 Chrome DevTools Protocol 寫成的端對端測試、隱私政策的靜態檢查、公開數字的自動核對，以及讓 CI 失敗能被看見的 annotation 機制。這些不是任何單一演算法，卻是一個零後端靜態站能被信任的原因。

目錄

摘要	1
1 導論	5
1.1 要解的問題	5
1.2 三條設計原則	5
1.3 本報告的範圍	6
2 系統架構	7
2.1 分層	7
2.2 一次作答的資料流	7
2.3 離線與載入	8
3 題庫與內容管線	9
3.1 規模與分佈	9
3.2 難度不是一個數字，是三軸	9
3.3 來源聲明與永久編號	10
3.4 等價題與模板	10
4 引擎一：答案判分	11
4.1 為什麼不做符號化簡	11
4.2 取樣點的選擇	11
4.3 容差	11
4.4 不定積分：比增量，不比值	11
4.5 其他答案型別	12
4.6 選擇題的誘答理由	12
4.7 輸入語法	12
5 標準答案本身的獨立驗算	13
5.1 做法	13
5.2 CI 門檻	13
6 引擎二：白話證明檢查	14
6.1 三分之二的誠實	14
6.2 正規化	14
6.3 十三種句型	14
6.4 代數鏈的驗證	14

6.5	推論引擎與骨架	14
6.6	怎麼知道它沒放水	15
7	引擎三：能力模型	16
7.1	要回答的問題	16
7.2	從標籤到技巧	16
7.3	參數與理由	16
7.4	熟練度與置信度	16
7.5	四個象限	17
7.6	個人化的遺忘曲線	17
8	規劃器與抽題器	18
8.1	配方	18
8.2	降級必須誠實	18
8.3	新手保護與橋池	18
8.4	每日一題與抽題去重	18
9	入門課程	19
10	手寫計算紙	20
10.1	iPad 上的四個陷阱	20
10.2	繪製管線	20
10.3	「有時候卡一下」	21
10.4	測試怎麼測像素	21
11	資料層與隱私	22
11.1	三層紀錄	22
11.2	手寫草稿與 IndexedDB	22
11.3	隱私承諾與它的測試	22
11.4	零後端的社群功能	23
12	驗證文化與 CI	24
12.1	四十一支驗證器	24
12.2	九支端對端測試	25
12.3	CI 上看不見的失敗	25
12.4	公開的數字必須是真的	26
13	效能預算與模組化	27
13.1	預算	27
13.2	撞頂就搬，不調數字	27
14	結語	28

A	常數表	29
B	詞彙表	30

導論

1.1 要解的問題

微積分的練習題網站很多，它們的共同做法是：題目存在伺服器、答案交給伺服器判、進度存在帳號裡。BuzzCalculus 從第一天就拿掉這三件事——沒有伺服器、沒有帳號、沒有任何一筆資料離開使用者的裝置。這不是省錢的妥協，而是產品定位：一個能在考前一小時打開就練、練完關掉不留痕跡、離線也能用的工具。

拿掉後端之後，三個原本理所當然的事變成問題：

1. 使用者打進來的 $2x^3$ 、 $x^3 \cdot 2$ 、 $(2x^3)$ 都得判對，而題庫存的是 $2x^3$ 。判分器必須在瀏覽器裡跑，而且不能誤判——判錯一題造成的信任損失，大於新增五十題的收益。
2. 「今天練什麼」需要一個能力模型，而能力模型需要作答歷史。沒有資料庫、只有 localStorage 的 5 MB，歷史要怎麼存、怎麼推導、怎麼在使用者中斷幾週之後不「失憶」。
3. 證明題沒有選項可選、沒有數字可比。瀏覽器裡跑不了 Lean，但「這一步有沒有跳」這件事，真的做不到嗎？

1.2 三條設計原則

純函式 kernel

src/kernel/ 底下 23 個模組、14,489 行，每一個都只吃 (records, problems, now) 這類輸入、吐資料。不碰 DOM、不碰 localStorage、不呼叫 Date.now()——時間是參數。這帶來兩個直接的好處：能力曲線可以「回放」到任何一天，所以不需要每天寫快照；同一份程式碼可以被 Node 的驗證器 require 進來逐題釘住。

每一條承諾都有測試

隱私政策上寫「關掉分析會停止收集」，半年後有人重構把那個判斷拿掉，政策就變成不實陳述，而且沒有任何測試會紅。所以隱私政策裡的每一條、README 裡的每一個數字、每一題的答案、規格文件裡的現況快照，都有對應的靜態檢查：改壞了 CI 就紅。

推薦必須說得出為什麼

使用者信任推薦系統的唯一來源，是它說得出理由。規劃器輸出的每一份配方都帶一句 why，它是產品的一部分而不是 debug 訊息，驗證器會擋住空字串。同樣的精神：抽不滿題目時要降級補滿並誠實說明，樣本不夠時能力數字寧可留白也不猜。

1.3 本報告的範圍

第 2 章講架構與資料流；第 3 章講題庫與內容管線；第 4 到第 8 章是三個引擎(判分與獨立驗算、白話證明、能力模型、規劃與抽題)；第 9 章入門課程；第 10 章手寫計算紙；第 11 章資料層與隱私；第 12 章驗證文化與 CI；第 13 章效能預算與模組化；第 14 章結語。附錄列出所有常數與驗證器。

系統架構

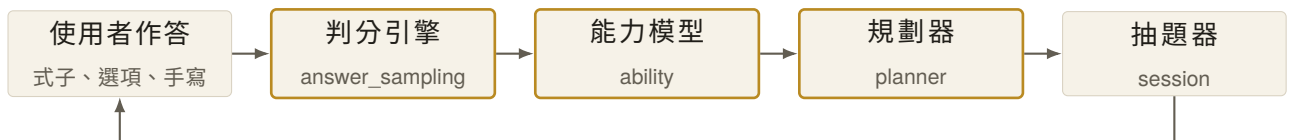
2.1 分層

整個站是一份 index.html、一份 styles.css、一支 service worker，和 65 個 `<script defer>`。沒有建置步驟，沒有 node_modules：TypeScript 只在開發與 CI 用 JSDoc 做型別檢查（`tsc --noEmit`），不進 runtime。

層	行數	內容
src/app.js	15,577	畫面、狀態、事件委派。單一 IIFE，正在逐段拆出去(第 13 章)。
src/kernel/*.js (23 檔)	14,489	純函式引擎與產生的側表：判分取樣、白話證明、能力模型、規劃器、抽題器、紀錄分層、手寫繪製、技巧圖譜、難度三軸、來源、永久編號、驗算側表。
src/problem*.js (37 檔)	—	題庫資料，995 KB。
src/course.js、share_cards.js、proof_lab_ui.js	—	從 app.js 搬出去的整段 render：入門課程、分享卡、證明訓練畫面。
tools/ (66 支)	—	41 支驗證器、9 支 E2E、產生器、剖析工具。零依賴，只用 Node 內建模組。

表 2.1: 程式的四層。kernel 與 app 的行數相當，這是刻意的：邏輯要能離開畫面被測。

2.2 一次作答的資料流



迴圈只經過 localStorage 與 IndexedDB；能力曲線由紀錄回放推導，不存快照

圖 2.1: 每一局訓練的迴圈。三個金框是本報告的主角。

1. 使用者送出答案，判分引擎回傳對錯與一句給人看的訊息（不是判分器的內部話）。
2. 作答寫進三層紀錄(第 11 章)：完整明細、精簡逐題 tuple、局摘要。
3. 能力模型從紀錄回放出每個技巧的熟練度、置信度、速度象限。
4. 規劃器據此產生配方：幾題到期複習、幾題弱點、幾題新技巧，以及為什麼。

5. 抽題器照配方抽，抽不滿就降級補滿，並把降級寫進 `meta.fallbacks`。

2.3 離線與載入

service worker 精確列出快取清單(101 個項目涵蓋 65 支 script)，`validate_offline_assets.js` 檢查 app 用到的每一個圖示都在自帶的圖示集裡、每一個資產都在快取清單裡；`validate_app_shell.js` 檢查 repo 裡每一個要出貨的 HTML 都被 CI 的 Assemble 步驟複製了。CACHE_NAME 必須以 app 版本號開頭、後面接日期或標籤，否則同一版內改資源使用者拿不到新檔(`validate_version.js`)。

題庫與內容管線

3.1 規模與分佈

題庫 1,994 題，只有微積分(理科題已於 2026 年 8 月移到姊妹站)。四個主題、六個難度等級：

主題	R1	R2	R3	R4	R5	R6	合計
極限	45	60	51	40	66	69	331
微分	53	116	132	167	111	110	689
積分	39	77	85	129	131	243	704
級數	33	42	45	25	60	65	270
合計	170	295	313	361	368	487	1,994

表 3.1: 題庫依主題與難度的分佈。R6 最多，因為競賽風格與名校風格的題包集中在那裡。

答案型別決定判分路徑：數值 1,356 題、式子 317、不定積分 142、文字判定 100、集合 17、區間 16、作圖表 22、互動圖形 24(點位 8、切線 4、選圖 12)。

3.2 難度不是一個數字，是三軸

每一題的難度由三個 1-3 的軸決定：步數(Steps)、冷門度(Obcurity)、負荷(Load，題幹與答案的 token 數加上是否需要動筆)。等級由三軸算出，公式在 `kernel/rubric.js`，瀏覽器與工具鏈共用同一份——曾經有過兩份，結果畫面上的 R5 和難度說明頁算出來的 R3 對不起來。

```
function rankFrom(axes, skillCount) {
  const raw = axes[0] + axes[1] + axes[2];          // 3..9
  let rank = RAW_TO_RANK[raw] || (raw <= 3 ? 1 : 6);
  // Obscurity at 3 with a high total: at least R5. Only this axis counts --
  // many steps means "takes time", not "cannot do".
  if (axes[1] === 3 && raw >= 7) rank = Math.max(rank, 5);
  // Chaining three or more independent skills: +1. Each step is easy;
  // the difficulty is thinking to connect them.
  if (skillCount >= 3) rank = Math.min(6, rank + 1);
  return rank;
}
```

設計決定

三軸表是從現行等級反推的，遷移當下每一題的等級完全不變。表裡標成人工複核過的題代表有人真的看過並負責；其餘的理由由程式依三軸生成，而且會明講「未經人工複核」。機器

推導的東西不能看起來像有人背書。

3.3 來源聲明與永久編號

每一題有一筆來源聲明：原創、改編、取材自某種風格、公有領域。`validate_content_metadata.js` 掃題幹、解說與來源欄位裡的名校與競賽字眼(大小寫敏感、字界比對——否則 `limit` 裡的 `mit` 會讓整個題庫誤報)，出現的題不准標成原創，而且來源註只准寫「某某風格的原創題」。2026 年 9 月起產品可見文字裡連競賽名稱都不出現。

每一題另有一個永不回收的永久編號(`kernel/uid_map.js`)。它存在的理由是對戰碼：碼裡裝的是題目的永久編號而不是隨機種子，題庫成長重排之後，兩台裝置解出來仍是同一份題。

3.4 等價題與模板

不同題包裡常有只差記法的同一題($\ln$ 與 $\log$ 、 $x^2/2$ 與 $\frac{x^2}{2}$)。刪任何一邊都會在那個題包上開一個洞，所以改成抽題時去重：`detect_duplicates.js` 產生等價側表(42 組，其中 3 組人工複核判定為誤判)，同一局或同一份考卷只會出現其中一題。另有模板題(`tmpl-*`)由參數展開，`expand_templates.js --check` 保證展開結果與版控一致。

引擎一：答案判分

4.1 為什麼不做符號化簡

判斷兩個式子等價的正統做法是符號化簡：把兩邊都化成標準形再比。瀏覽器裡沒有電腦代數系統，自己寫一個又會在三角恆等式、對數律、絕對值上無窮無盡地補洞。BuzzCalculus 選了另一條路：在一組固定的取樣點上求值，逐點比較。這是有限取樣，不是形式證明——程式碼第一行就寫著這句話——但它的失敗方式是可預測的，而且可以用取樣點的選擇把誤判壓到實務上看不到。

4.2 取樣點的選擇

```
// Positive-only samples had a real hole: sqrt(x^2) and x agree on every
// positive point but are not the same function. Negatives separate them.
// "Nice" numbers (0.5, 1, 2) are where different functions collide most
// often (sin(pi/6) = 1/2 and friends), so the points sit near irrationals.
const SAMPLES = [0.3137, 0.7211, 1.2345, 1.9871, 3.3013,
                  -0.6180, -1.3247, -2.1069];
```

單變數用這 8 個點；多變數用線性同餘產生 24 個點，每個座標獨立產生，避免全落在 $y = x + c$ 一條直線上。題目可以帶定義域約束（例如 $x > 0$ ），取樣點先過濾；至少要有三個點兩邊都有定義才算「判得穩」，否則回「格式讀不穩」而不是判錯。

4.3 容差

$$|a - b| \leq \max(10^{-6}, 10^{-5} |a|)$$

相對容差 10^{-5} 對手算的式子綽綽有餘，同時足以分開 $\sin x$ 與它的三階 Taylor 多項式在 $x = 1.9871$ 的差。

4.4 不定積分：比增量，不比值

反導函數差一個常數也算對，所以不能直接比 $F(x)$ 。取第一個取樣點當基準，比較每一點相對基準的增量：

$$|(b_i - b_0) - (a_i - a_0)| \leq \max(10^{-5}, 10^{-5} |a_i - a_0|) + 4\varepsilon_{\text{mach}} \max(|b_i|, |b_0|)$$

容差由標準答案的變化量決定，不能被使用者寫進去的巨大常數放大；最後一項是浮點捨入的保護——常數大到捨入誤差超過容差時，明講「積分常數太大，無法可靠比較」。

題目	學生輸入	判定
$\int 6x^2 dx$	$2x^3 + 100$	正確 ：原函數相差常數
$\int 6x^2 dx$	x^3	錯 ：微分後不相同，附「差一個 1/3」的提示
$\frac{d}{dx}\sqrt{x}$	$1/(2*\text{sqrt}(x))$ 與 $0.5*x^{(-0.5)}$	正確 ：多點代入等價
$\lim_{x \rightarrow 0} \frac{\sin x}{x}$	$1 \div 1.0 \div 2/2$	正確

表 4.1: 判分引擎的實際行為。訊息一律是給人看的話，不是判分器的內部話。

4.5 其他答案型別

- **集合**：元素用逗號隔開，順序不影響判分，每個元素各自走數值比較。
- **區間**：端點用 () 或 []，開閉有差；聯集用 U；無窮寫 inf。格式說明在輸入框旁邊先講清楚，不讓使用者為了猜格式重打。
- **文字判定**：收斂、發散、條件收斂、絕對收斂、不存在，含中英別名。
- **互動圖形**：點位題比較點到目標的距離，切線題比較拖出的斜率與正解，都有容差且兩端都判。
- **作圖表**：每格一個判分，錯的選項都要具名說明。

4.6 選擇題的誘答理由

選擇題答錯時，「在 $x = 0.31$ 代入時不相同」是判分器的話，不是給人看的。實作改成：告訴使用者選了哪一個、它像是怎麼錯的（誘答理由，例如「指數減 1 忘了」），正解在下面。誘答選項在產生時已經用判分器濾掉與正解等價的寫法，否則會出現兩個都對的選項。

4.7 輸入語法

答案輸入採 WebWork 式語法 (* 乘、^ 次方、sqrt、log 即 ln)。輸入框有即時 LaTeX 預覽，回答的是「我打的被讀成什麼」——那是送出前唯一能自我檢查的地方。手機上的數學鍵盤依題型排序：積分題先看到 ^ / () sqrt，極限題先看到 pi e inf。

標準答案本身的獨立驗算

判分引擎只能保證「跟標準答案一致」。標準答案錯了怎麼辦？在 `verify_answers.js` 之前，答案的正確性完全靠人工檢查——也就是完全沒有機器把關。

5.1 做法

從題幹推出一條與解題無關的數值路徑，再跟標準答案比：

- 導數：對題幹的函數做數值微分，與答案的式子在取樣點比。
- 不定積分：把答案微分回去，與被積函數比。
- 定積分、極限、級數：直接用數值方法算出來，與答案的值比。
- 專用驗算：收斂半徑、Taylor 係數、Hessian 行列式、多變數極限、留數、Wronskian、區域積分、參數曲線、卷積、Beta 與 Gamma 積分、常微分方程的殘差……共 90 餘種方法。

方法	題數	方法	題數
定積分	303	收斂半徑	41
極限	257	級數收斂判定	42
導數(式子)	202	參數積分	36
不定積分	142	Taylor 係數	33
級數和	131	線積分	26
Taylor 導數	57	區域積分	18
數值微分(點值)	32	一階常微分方程	16

表 5.1: 驗算方法的分佈(前 14 種)。全部 1,753 題通過，241 題目前無法獨立驗算(多為文字判定與概念題)。

5.2 CI 門檻

`--ci` 模式下兩件事會讓建置失敗：不符數大於零，或覆蓋率低於門檻。驗算結果寫成側表 `kernel/verified_answers.js`，題庫頁上每一題有「答案已驗算」的標記——這是競品沒有的一句話，藏在 CI 裡等於沒有。

危險的是安靜地算錯

驗算器最危險的失敗方式不是「算不出來」，是「安靜地算錯」：一支方法對某類題幹解析錯了但仍回傳一致，覆蓋率好看、其實沒驗。所以黃金測試(`validate_golden.js`)錄下每一支驗算方法在固定題目上的中間結果，改動解析器就會紅。

引擎二：白話證明檢查

6.1 三分之二的誠實

瀏覽器裡跑不了 Lean，Lean 只在 CI 裡驗 8 題經典證明。但「這一步有沒有跳」這件事，做得到三分之二——前提是老實分清楚哪三分之二：

1. **句型引擎**：認得這句話在做什麼（設變數、假設、代數推導、引用定理、分情況、歸納、下結論）。
2. **代數引擎**： $A = B \leq C < D$ 這種鏈，在目前的假設下隨機取點驗——等式看差、不等式看方向。
3. **推論引擎**：一小本規則字典（均值定理、三角不等式、夾擠……），比對「你寫的句子」與「規則會吐出的形狀」。

剩下的三分之一——存在句、抽象函數 f 、數論——標黃：看得懂但驗不了，不假裝。

6.2 正規化

使用者會混用全形標點、希臘字母、LaTeX 指令與 Unicode 符號。第一步把它們收斂成一種寫法：全形數字與字母轉半形；「，。：；（）」轉半形； $\leq, \geq, \neq, -, \times, \cdot$ 轉成 ASCII； ε 與 ϵ 都變 `eps`； $\sqrt{}, \pi, \infty$ 各有對應； $|x-2|$ 這種不巢狀的絕對值翻成 `abs(x-2)`。

6.3 十三種句型

6.4 代數鏈的驗證

一行則 $|3x - 6| = 3|x - 2| < 3\delta = \varepsilon$ 被拆成四段；在目前所有假設 ($0 < |x - 2| < \delta, \delta = \varepsilon/3$) 下隨機取一組滿足假設的變數值，逐段驗：等式看兩邊差、不等式看方向。方向寫反的那一段標紅，並回傳一組帶數字的反例。

6.5 推論引擎與骨架

規則字典裡每一條定理有「需要的前提形狀」與「會吐出的結論形狀」。寫「由平均值定理，存在 $c \in (x, y)$ 使 $f(y) - f(x) = f'(c)(y - x)$ 」會被認出來；接著若跳過「因為 $f'(c) = 0$ 」直接寫「故 $f(x) = f(y)$ 」，結論不放行。每一題另有骨架檢查：一份 ε - N 證明至少要有任取、取 N 、假設 $n > N$ 、推導、結論五個角色，缺角色的證明就算每一行都綠也不算完成。

句型	標籤	觸發的寫法(節錄)
let	任取／設／取	任取、任意取、給定、固定、設、令、取、let、fix、choose
assume	假設／若…則…	假設、若、如果、suppose、assume、if
by	由〈規則〉，…	由、根據、依據、利用、by、using、applying
because	因為…，所以…	因為…，所以／故／因此
claim	則／所以〈推導〉	則、那麼、得、所以、故、因此、於是、即、同理
cases / case	分情況、情況 N	分兩種情況、考慮三種情況、case 1
induction	用歸納法	用數學歸納法、對 n 做歸納
base / hypothesis	當 $n = 1$ 時、假設 $n = k$ 時成立	當 $n = 1$ 時、假設 $n = k$ 時
contradiction-start / contradiction	反設、矛盾	反設、假設不然、suppose not；… 矛盾
qed	得證	得證、證畢、Q.E.D.、■
unknown	—	一行有兩句、或只有式子沒有句型

表 6.1: 句型引擎的分類。一行只能一句：一行裡有兩個句號就直接判成讀不懂，並說為什麼。

行	內容	判定
1	任取 $\varepsilon > 0$ 。	綠
2	取 $\delta = \varepsilon$ 。	綠(設定本身沒有錯)
3	假設 $0 < x - 2 < \delta$ 。	綠
4	則 $ 3x - 6 = 3 x - 2 < 3\delta = \varepsilon$ 。	紅： $3\delta = \varepsilon$ 不成立，反例 $\varepsilon = 0.3, \delta = 0.3$
5	所以 $\lim_{x \rightarrow 2} 3x = 6$ 。	黃：結論沒有支撐

表 6.2: 一份 δ 取錯的 ε - δ 證明。紅在第 4 行，不是第 2 行——第 2 行的設定本身不是錯，錯的是它撐不起第 4 行。

6.6 怎麼知道它沒放水

validate_proof_lang.js 對 43 題的參考證明做三種變異：刪一行、改一個符號、換一個方向。參考證明必須全綠；每一個變異版必須紅在正確的那一行——不是「有紅就好」。另外 68 條經典證明(步驟重排、填空決策點、自評)中有 8 條屬 Lean 層，其參考解在 CI 裡用 Lean 4 編譯，編不過就擋部署；其餘證明裡的定量宣稱(「這個積分等於 $\pi/4$ 」)由 verify_proof_claims.js 數值驗算。

標黃而不是標綠

存在句、抽象函數、數論命題驗不了。最省事的做法是放行，最誠實的做法是標黃並寫「這一行看得懂但驗不了」。選了後者，代價是一份完全正確的證明可能不會全綠——說明頁直接告訴使用者這一點，而不是讓黃色看起來像錯。

引擎三：能力模型

7.1 要回答的問題

別人給正確率；這裡要分得出「不會」「來不及」「壓力下垮掉」三種。同樣是 60% 的正確率，一個人是每題都算到最後一秒才錯，另一個人是三秒就送出——處方完全不同。

7.2 從標籤到技巧

題庫有 324 個標籤，但那不是能力模型能用的座標系：標籤混了技巧、題型、題包來源、難度標記。kernel/skill_graph.js 把標籤收斂成 80 個技巧節點並定義前置關係。一個技巧由「標籤命中且主題相符」決定——taylor 這個標籤同時出現在極限題與級數題，必須靠主題分開，否則極限題會被算成級數技巧。

7.3 參數與理由

常數	值	為什麼
權重時間常數	30 天	30 天前的一題約值今天的 0.37 題
先驗強度	6	等價於 6 次「不確定」的作答
先驗正確率	0.45	偏低：沒證據就不給高分
測準門檻	加權量 12	到這裡才算「測準了」；置信度低於 0.4 一律顯示「未測」而不是分數
分流最低樣本	8 題	用原始題數，因為要能直接講給使用者聽
看過解答才對	0.5	只給半分
提示扣分	每層 0.15，下限 0.55	提示是拐杖，但不是零分
「快」的門檻	$\text{elapsed} / \text{timeLimit} < 0.6$	相對题目的時限，不是絕對秒數
「準」的門檻	正確率 > 0.7	

表 7.1: 能力模型的常數。每一個都要能解釋，否則就會變成沒人敢動的魔術數字。

7.4 熟練度與置信度

每筆作答的權重 $w = e^{-\Delta t/30 \text{ 天}}$ ，分數 $s \in [0, 1]$ 依對錯、是否看過解答、用了幾層提示調整。技巧的熟練度以先驗平滑：

$$\text{mastery} = \frac{\sum w_i s_i + 6 \times 0.45}{\sum w_i + 6}, \quad \text{confidence} = \min\left(1, \frac{\sum w_i}{12}\right)$$

置信度不到 0.4 就顯示「未測」。這是產品層的決定：寧可留白，也不要給錯的能力數字。

7.5 四個象限

用「快不快」與「準不準」把每個技巧放進四格，每一格對應不同處方：

象限	快	準	處方
反射區	是	是	拉高難度或進 Boss
會但慢	否	是	同技巧限時訓練，時限逐次縮短
衝太快	是	否	練習模式 + 標註錯因
還沒建立	否	否	看關鍵一句 + 三層提示 + 慢練

表 7.2: 速度象限。「你會做，是壓力下垮掉」這句診斷來自限時與不限時正確率的差。

7.6 個人化的遺忘曲線

30 天是預設的時間常數，但每個人忘得不一樣快。當一個人有 12 筆以上「隔多天回訪同一技巧」的實績，模型用回訪時的表現與預測的偏差校正半衰期倍率，範圍 0.6 到 1.8。首頁會說「依你 n 次隔多天回訪的實績，你忘得比模型預設慢」。

回放，不存快照

能力曲線的每一個歷史點都由紀錄與當時的時間重新算出，所以成長曲線不需要每天寫快照，使用者中斷三週再回來也不會斷線。代價是每次算要掃紀錄——第 11 章的分層儲存讓這件事在 5,000 筆逐題 tuple 上仍是毫秒級。

規劃器與抽題器

8.1 配方

規劃器輸出的不是題目，是配方：每個角色佔多少比例，加上一句為什麼。三種基本配方：

情境	角色與比例		
維持手感	到期複習 0.6	一般練習 0.4	
日常訓練	到期複習 0.35	弱點 0.4	新技巧 0.2 • 拉高難度 0.05
新使用者	暖身 0.2	弱點 0.3	新技巧 0.3 • 難題 0.15

表 8.1: 配方的角色比例。考前衝刺另有依考試日期與範圍調整的配方。

8.2 降級必須誠實

一個新使用者沒有錯題可複習，一個只練過極限的人沒有級數的弱點資料，難度上限開 2 的人抽不到 R5。這些情況下配方一定有角色抽不滿——那時候必須降級補滿，不能留白。給使用者 12 題就是 12 題，少給 3 題比給錯 3 題更傷信任。降級路徑寫在 `meta.fallbacks`，畫面據此說「今天沒有到期的錯題，改多練兩題弱點」。

補位題掛在錯的名下

第一版對全新使用者顯示「7 到期複習 8 弱點」——那是補位規則填出來的數字，對一個什麼都沒做過的人是假的。現在補位題不掛在任何角色名下，而且新手的第一份訓練固定 8 題、由淺入深。

8.3 新手保護與橋池

剛從入門課程畢業的人直接丟進主線，第一題可能就是有理化或反三角——沒教過，信心當場塌掉。所以有一個「橋池」：校準後等級為 1、三個單元、不是文字判定題、而且沒有課裡沒教的標籤，131 題。畢業關從橋池抽 10 題、8 題過；畢業前與畢業後頭三局，任何訓練的池子都縮到橋池；頭四局不倒數。驗證器另外守橋池大小、課裡用的題全在橋池、排除的標籤真的存在於題庫（改名了就會安靜地漏進去）。

8.4 每日一題與抽題去重

每日一題由日期決定種子、全站同一題、一天一次；成績可以出成圖卡分享。所有抽題都經過等價側表去重，同一局不會出現只差記法的兩題。

入門課程

完全沒學過微積分的人打開站台會被 1,994 題淹死。入門課程補的是那一層：十六課，順序照一般課本——函數、極限與連續、微分、積分。

單元	課
函數	函數是什麼(理)、常見的函數家族與合成(理)
極限與連續	極限是什麼、代進去變 $0/0$ 怎麼辦、 x 跑到無窮遠、三個要背的極限、連續與中間值定理(理)
微分	導數就是斜率、微分公式、乘法律與除法律、連鎖律、 f' 的正負與極值(理)、均值定理(理)
積分	積分是微分的反操作、定積分是面積、黎曼和與基本定理(理)

表 9.1: 十六課。標「理」的六課是理論課。

兩種課：計算課是白話概念、一題來自題庫的逐步示範、小測(選錯會說為什麼)、三題不倒數的練習；理論課沒有題庫練習(題庫裡沒有、也不該有「定理說了什麼」這種反射題)，示範是一段推導，小測至少三題，全對就算完成。逐步示範揭下一步之前會先讓人猜：真的下一步與同一題別的步驟二選一，猜錯照揭，只多一句「其實是這個」。每課完成後有一題同單元的 R2 延伸挑戰。

`validate_course.js` 守：示範與練習必須是題庫裡的 R1 題、小測恰好一個正解且錯的選項都寫理由、課文裡「第 n 課」的 n 不超過總數、正解不能六成以上都放第一個、課文的每一條 TeX 都用本地 KaTeX 嚴格模式排一次。

手寫計算紙

10.1 iPad 上的四個陷阱

真正的使用情境是 iPad 加 Apple Pencil，那個情境有四個瀏覽器預設行為會直接毀掉它：長按選取（手掌碰到題目就整段反白、跳出拷貝選單）；手掌本身是 touch 事件，跟筆同時發生；Pencil 取樣 120–240 Hz 而 pointermove 一個 frame 只觸發一次，不撈 `getCoalescedEvents()` 就是多邊形；每次 move 重畫整塊板子是 $O(n^2)$ ，寫到第三行就看得出幀。

10.2 繪製管線

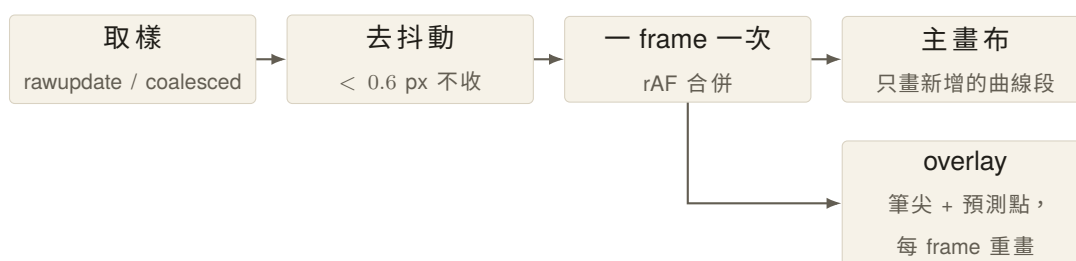


圖 10.1: 一筆從取樣到畫面。收筆時主畫布補到真正的筆尖，overlay 清空。

- **平滑**：以相鄰兩個取樣點的中點為端點、取樣點本身為控制點畫二次曲線——這是把折線變成手寫感最便宜的做法，而且逐段可畫。第一段要從真正的起點出發，否則落筆的圓點跟線條之間有一小段空白。
- **增量**：用 `stroke.drawnTo` 記進度，只畫還沒畫過的那一段。
- **筆尖**：二次曲線要知道下一個取樣點才畫得出來，已提交的曲線永遠停在倒數第二個點。原本每個取樣都補一段直線到筆尖、下一個取樣再用曲線蓋掉——筆尖每秒抖兩百多次、繪製量三倍。現在筆尖那一段畫在 overlay，每 frame 清掉重畫，並接上瀏覽器的預測座標（Chrome 與 Android 有，Safari 沒有），墨水比筆尖早半步到。
- **寬度**：有壓感的筆用壓力，往回平均三點，落筆與收筆墊高（否則每一筆的尾巴收成針尖，一條分數線變成透鏡形）；沒有壓感的裝置用速度，作用範圍與壓感路徑相稱。
- **手掌**：看到筆就記時間，1.4 秒內的觸控一律忽略。
- **橡皮擦**：用 `destination-out` 真的挖掉墨水；拿背景色蓋過去會把方格紙的格線一起蓋掉。畫布保持透明，紙的方格與黑板底色都是 CSS。

10.3 「有時候卡一下」

真兇不是繪製

使用者回報「有時候卡一下」。用 CDP Profiler 畫 40 筆、每筆 120 個取樣，主執行緒忙 1,564 ms，其中 `loadRecords` 151 ms——比繪製本身 (`paintStrokeTail` 68 ms) 還多。原因：每問一次筆寬或紙型就從 `localStorage` 讀整包紀錄再 `JSON.parse`，而繪製每 frame 都問。紀錄越大越卡。改成快取設定、存檔時清掉之後，`loadRecords` 從熱點消失，主執行緒忙 1,175 ms。

教訓寫進規則：畫面上任何每 frame 跑的東西都不准呼叫 `loadRecords`；再有人說卡，先跑 `tools/profile_board.js` 看長任務數與前十名自身時間，不要猜。

10.4 測試怎麼測像素

`e2e_handwriting.js` 有 45 條斷言，全部打在畫布的像素與實際派發的 `PointerEvent` 上：手掌畫不出線、長按不跳選單、橡皮擦不蓋格線、復原不會整頁重繪、還在寫的時候墨水就已經到筆尖、每個取樣點只被畫一次(上線後的當機就是兩個監聽器都掛、每個取樣畫兩次)、寫過之後計算紙收得起來。合成事件不會自帶 `coalesced` 批次，測試手動補上——不補的話那條路徑在測試裡永遠不會發生，而那正是它漏到線上的原因。

資料層與隱私

11.1 三層紀錄

能力模型需要長時間的作答歷史，但完整的局明細上限只有 40 場——一個認真練的使用者兩週就洗掉了，等於每兩週失憶一次。解法是分三層，各自負責不同的時間尺度：

層	上限	用途	大小
history	40 場	完整局與每題明細：結算頁、最近表現、明細檢視	
attemptLog	5,000 筆	極精簡逐題 tuple(不是物件)：能力模型、趨勢、百分位	約 250–300 KB，半年份
sessions	400 場	局級摘要：熱力圖、成長曲線	涵蓋一年以上

表 11.1: Records v2。同一把 localStorage key、同一支正規化函式，只加頂層命名空間。

舊使用者第一次載入時從 history 回填 attemptLog，不需要遷移腳本，也不會看到空白的趨勢圖。validate_records_v2.js 驗：history 截斷後能力模型仍保住 480 筆（只讀 history 只剩 96 筆）；兩台裝置的紀錄合併滿足交換律與冪等（80 + 60 → 140 筆，衝突以 updatedAt 新者為準）。

11.2 手寫草稿與 IndexedDB

筆畫陣列一題就能到幾十 KB，存 localStorage 會擠掉紀錄，所以草稿進 IndexedDB。存的理由只有一個：重做錯題時，「我上次是哪一步算錯的」這個問題的答案就在上次的草稿裡——上次答錯的草稿預設攤開對照，答對的收起（攤開就變照抄）。

11.3 隱私承諾與它的測試

政策上寫的每一條在 validate_privacy.js 裡都有靜態檢查：

- 關掉分析不只是不送事件，連 gtag.js 都不載——光是載入那支 script 就已經對 Google 發出請求；檢查器確認 analyticsEnabled() 的判斷在載入 script 之前。
- 清除資料掃的是整個 buzzcalculus. 命名空間，不是一份固定清單——自動備份的 key 帶時間戳，固定清單永遠列不完；E2E 曾抓到「按了清除，一份完整備份還躺在 localStorage 裡」。
- 錯誤遙測只送型別、檔名、行號，不送訊息與堆疊——JS 引擎組出來的訊息常把變數的值印進去，那可能是題目或使用者的答案。
- 回報題目不能只寫進使用者自己的瀏覽器：那比沒有回報按鈕更糟，它製造了「已經反映過」的錯覺。

11.4 零後端的社群功能

對戰碼 BZD1:417.882.⋯~8.225 裝的是題目的永久編號與發戰帖者的成績，貼給朋友、對方貼碼應戰、結算頁直接分勝負，不用帳號不用網路。成就分享卡與每日一題成績卡在本機 canvas 畫成 PNG，手機走系統分享(navigator.share 帶檔案)，桌機下載——資料一律不出裝置。

驗證文化與 CI

12.1 四十一支驗證器

表 12.1: 驗證器與它守的東西。全部只用 Node 內建模組。

驗證器	守什麼
verify_answers	從題幹獨立驗算標準答案，不符必須為零
validate_problems	題目欄位完整、答案格式合法、時限與難度存在
validate_answer_checker	判分器對固定的正反例判得對
validate_content_metadata	永久編號、三軸、來源聲明齊全；名校字眼不准標原創
validate_calculus_only	題庫必須是純微積分
validate_katex	每一題的題幹真的拿 KaTeX 渲染一遍
validate_hints	/ 提示品質、推導式提示重新驗證、選圖題與作圖表的把關
derived_hints	/
graph_choices	/ work-sheets
validate_daily_one	每日一題釘選表；歷史可回放
validate_training_packs	題包非空、主線每一關抽得出題、抽題規則
/ path_nodes	/ problem_selection
validate_skill_graph	80 個技巧節點與前置關係一致
validate_ability_model	能力模型在合成紀錄上的行為(衰減、留白、象限)
validate_planner	配方帶 why、降級路徑、等價去重
validate_records_v2	分層截斷保住資料、合併交換律與冪等
validate_proof_lang	參考證明全綠、三種變異紅在正確的行
verify_proof_claims	經典證明裡的定量宣稱數值驗算
validate_course	入門課程的示範、練習、小測、TeX、橋池
validate_golden	黃金檔：判分、繪圖指令序列、驗算方法的中間結果
validate_analytics	事件表與程式碼一致、不上報使用者輸入
validate_privacy	政策裡的每一條承諾
validate_offline_assets	/ 快取清單、出貨頁面、版本號
app_shell	/ version
validate_performance_budget	app.js、kernel、樣式、題庫、script 數各有預算
validate_public_claims	README 的 21 個數字從程式碼算出來

驗證器	守什麼
spec_snapshot	規格文件的現況快照必須與程式碼一致
smoke_app_render	在 Node 裡 render 一次首頁
regen_all -check	產生器的產物與版控同步

12.2 九支端對端測試

E2E 不用 puppeteer：Node 22 內建 WebSocket 與 fetch，而 Chrome DevTools Protocol 就是「連上 ws、送 JSON」，tools/lib/cdp.js 兩百多行就夠。斷言原則：只斷言使用者看得見、點得到的東西——文字要有面積且 opacity 不是 0，互動要真的觸發事件委派，console 不能有錯、資源不能有 404。其他驗證器比對的是 render 出來的 HTML 字串，那一層看不到「畫面是白的」——實際抓到過結算頁 DOM 有兩萬字、畫面零字。

套件	斷言	走的路
main_flow	34	開局、答題、結算、錯題進本、四個分頁、快捷鍵、清除資料
storage	16	localStorage 額度爆掉時的救援、匯出、離線重載
handwriting	45	畫布像素與 PointerEvent (第 10 章)
mobile	41	390×844 真視窗：橫向溢出、觸控目標 44 px、iPad 鍵盤
layout	35	外殼幾何：分頁列貼底、側欄不蓋內容
responsive	93 態	9 種尺寸 × 9 頁
proof_lang	37	教學到題目、打字即時三色、提交紀錄
beginner	72	定位、新手保護、十六課、畢業關、橋池
graph_interactive	10	SVG 上的點位與拖曳

表 12.2: 端對端測試。本機用 e2e_all.js 並行跑，九支約十分鐘。

12.3 CI 上看不見的失敗

GitHub Actions 的 job log 要認證才抓得到，而 check-run 的 annotations 走公開 API 就讀得到。所以每一條失敗的斷言都印成 ::error title=…:…，30 支驗證器由一支執行器逐支跑、紅的那支變成 annotation。「CI 紅、本機綠」時，這是唯一能看到 runner 上哪條倒了的方法。

CI 從 9 月 1 日紅到 9 月 16 日

部署與測試在同一個 job，任何一支 E2E 紅站台就不更新，兩週的工作使用者一個都沒看到。五支 E2E 全是同一種病：sleep(500) 這種「開發機剛好夠」的固定等待，在慢好幾倍的 runner 上輸給 debounce、rAF render 與 950 ms 的自動前進。修法一律是「輪詢等元素真的出現」，斷言本身不改；另外加了 CPU 節流開關在本機重現 runner 的節奏。還有兩個假陽性：離線重載其實只是改 hash 沒有真的重載；結算頁的彩帶粒子飛完停在 opacity 0，被當成「內容被藏住」。

12.4 公開的數字必須是真的

README 裡的題數、驗算題數、技巧數用標記包起來，`validate_public_claims.js` 從程式碼算出來比對；規格文件開頭的現況快照(題數、`app.js` 行數、驗證器數)由 `spec_snapshot.js` 產生並在 CI 檢查一致。數字錯掉的話，讀的人會拿它當事實去做取捨。

效能預算與模組化

13.1 預算

項目	現況		預算	使用率
app.js 主程式	722 KB	720 KB → 搬模組後回到預算內		—
kernel 邏輯模組	271 KB		280 KB	97%
題庫資料	995 KB		2,400 KB	41%
styles.css	305 KB		310 KB	98%
其他 src 腳本	172 KB		300 KB	57%
script 標籤	65		65	100%

表 13.1: 效能預算。撞頂時不調數字，搬整段出去；要調就寫清楚為什麼。

13.2 撞頂就搬，不調數字

這一輪兩次撞到 app.js 的 720 KB：第一次把週報與成就分享卡（230 行 canvas 程式）搬成 share_cards.js；第二次把錯因處方、段考預測、模式推薦三支純函式搬進 kernel/planner.js，app 端的依賴用 deps 注入。兩個坑：搬出去的東西若被 app.js 的 hooks 物件用簡寫引用（weeklyShareData,），文字搜尋抓不到，要靠型別檢查；模組在 IIFE 裡的 create 呼叫執行時，其後才宣告的 const 還在暫時死區，站台 URL 這種常數要用 getter 傳。

結語

這個專案裡真正難的東西，都是「在沒有後端的限制下把一件本來要靠伺服器的事做對」：判分、證明、能力模型、手寫。但讓它能被信任的不是任何一個引擎，而是把每一條承諾綁進 CI 的習慣——答案錯一題會紅、政策說謊會紅、README 的數字錯會紅、畫面白掉會紅。

還沒做的事：QR code 搬紀錄（紀錄幾十 KB 塞不進 QR，要先壓縮加分片）；app.js 拆首屏（目前的策略是撞預算就搬整段，總有一天要正面處理）；白話證明對存在句與抽象函數的三分之一；能力模型的遺忘曲線需要更多回訪實績才能個人化。這些都寫在規格文件裡，不是空話。

常數表

常數	意義	值
取樣點	判分引擎的單變數取樣	0.3137, 0.7211, 1.2345, 1.9871, 3.3013, -0.6180, -1.3247, -2.1069
多變數取樣	線性同餘產生	24 點，種子 20260914
判分容差	相對	$\max(10^{-6}, 10^{-5} a)$
判得穩	至少幾點有定義	3
權重時間常數	能力模型	30 天
先驗	強度 / 正確率	6 / 0.45
測準門檻	加權作答量	12
顯示門檻	置信度	0.4
提示扣分	每層 / 下限	0.15 / 0.55
遺忘倍率	範圍 / 最少樣本	0.6–1.8 / 12
history / attemptLog / sessions	上限	40 / 5,000 / 400
橋池	校準後 R1、三單元、非文字題、無未教標籤	131 題
畢業關	題數 / 過關	10 / 8
新手保護	不倒數的局數 (沒學過 / 高中先修 / 大一)	4 / 3 / 1
手掌視窗	看到筆之後忽略觸控	1,400 ms
去抖動	相鄰取樣距離	0.6 px
壓感平均	往回幾點	3
自動前進	答對之後	950 ms
CI 等待倍率	固定 sleep 乘	3

詞彙表

R1–R6

難度等級，由三軸(步數、冷門度、負荷)算出。R1 暖身、R2 基礎、R3 標準、R4 進階、R5 Boss、R6 Boss+。

技巧(skill)

能力模型的座標，80 個，由 324 個標籤收斂而成，有前置關係。

配方(recipe)

規劃器的輸出：每個角色佔多少題，加一句為什麼。

降級(fallback)

抽題器抽不滿某個角色時的補滿路徑，記在 `meta.fallbacks`。

橋池

入門課程與主線之間的題池：只出課裡教過的 R1 題。

三色

白話證明的逐行判定：綠(驗過成立)、紅(驗過不成立)、黃(看得懂但驗不了)。

黃金測試

錄下固定輸入的完整輸出序列，任何改動都要比對。

側表

由產生器從題庫算出、進版控的資料表：永久編號、三軸、來源、驗算結果、等價組。

annotation

GitHub Actions 的 `::error` 標記，不用認證就能從公開 API 讀到。